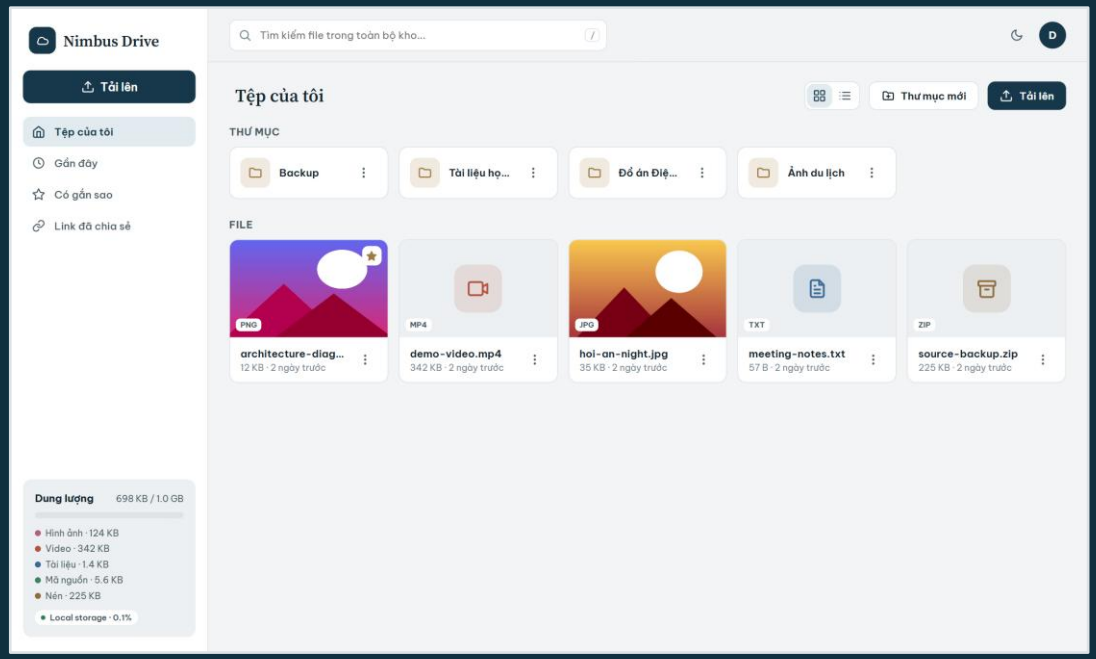
Nimbus Drive

Nền tảng lưu trữ file đám mây trên kiến trúc tương thích Amazon S3

Design and Deployment of a Spring Boot Containerized Cloud File Storage Platform based on AWS S3-Compatible Architecture with an Automated CI/CD Pipeline



Spring Boot 3.5 · PostgreSQL · S3 · Docker · Terraform · GitHub Actions

Sinh viên: [Họ tên] · MSSV: [MSSV]
GVHD: [Họ tên giảng viên] · Môn Điện toán đám mây

Nội dung trình bày

01

Bài toán & mục tiêu

Vì sao cần, làm gì

02

Kiến trúc hệ thống

S3-compatible, phân lớp, CSDL

03

Sản phẩm & demo

Tính năng, giao diện

04

Bảo mật

Các mối đe dọa và biện pháp

05

Container & CI/CD

Docker, Compose, GitHub Actions

06

Triển khai AWS & kết quả

Terraform, kiểm thử, hướng phát triển

Bài toán: lưu trữ dữ liệu linh hoạt, an toàn, tự động

Thực trạng

- Nhu cầu lưu trữ & chia sẻ file trực tuyến tăng mạnh
- Object storage chuẩn **S3** là nền tảng của mọi cloud drive
- Triển khai thủ công dễ lỗi, khó tái tạo môi trường
- Phụ thuộc một nhà cung cấp gây rủi ro (vendor lock-in)

1

Ứng dụng web hoàn chỉnh

Upload, thư mục, xem trước, chia sẻ, quota

2

Kiến trúc tương thích S3

Chạy trên AWS S3 hoặc S3 tự host, không đổi mã

3

Container hóa + IaC

Docker Compose & Terraform cho AWS

4

CI/CD tự động

Test → image → scan → deploy → smoke test

Khảo sát: khoảng trống mà đề tài lấp đầy

Tiêu chí	Google Drive / Dropbox	Nextcloud	S3 Console	Nimbus Drive
Mô hình	SaaS đóng	Tự host (PHP)	Công cụ quản trị	Tự host hoặc AWS
Kiểm soát dữ liệu	Thấp	Cao	Cao	Cao (S3 / SeaweedFS)
User, quota, thư mục	Có	Có	Không	Có
Chia sẻ link có hạn	Có	Có	Không	Có, thu hồi được
Presigned URL tải trực tiếp	Không công khai	Tùy cấu hình	Có	Có (15 phút)
Container + IaC + CI/CD	Không áp dụng	Cộng đồng	Không áp dụng	Có sẵn, đã chạy thật

Định vị: hệ thống tham chiếu gọn (~2.000 dòng Java) để học và áp dụng kỹ thuật cloud, không phải bản sao Google Drive

KIẾN THỨC NỀN

Nền tảng: tám khái niệm của Amazon S3

1

Bucket

Thùng chứa tên duy nhất toàn cầu, gắn với một region

2

Object key

Tên phẳng của object. Ở đây: owner/<uuid>, không dùng tên file

3

Presigned URL

URL ký sẵn SigV4, tự hết hạn: ủy quyền tạm thời cho trình duyệt

4

Versioning

Giữ nhiều phiên bản; lifecycle xóa bản cũ sau 30 ngày

5

SSE-S3

Mã hóa phía server AES-256, bật mặc định bằng Terraform

6

Block Public Access

Chặn mọi cấu hình công khai; chỉ IAM hoặc presigned URL

7

Strong consistency

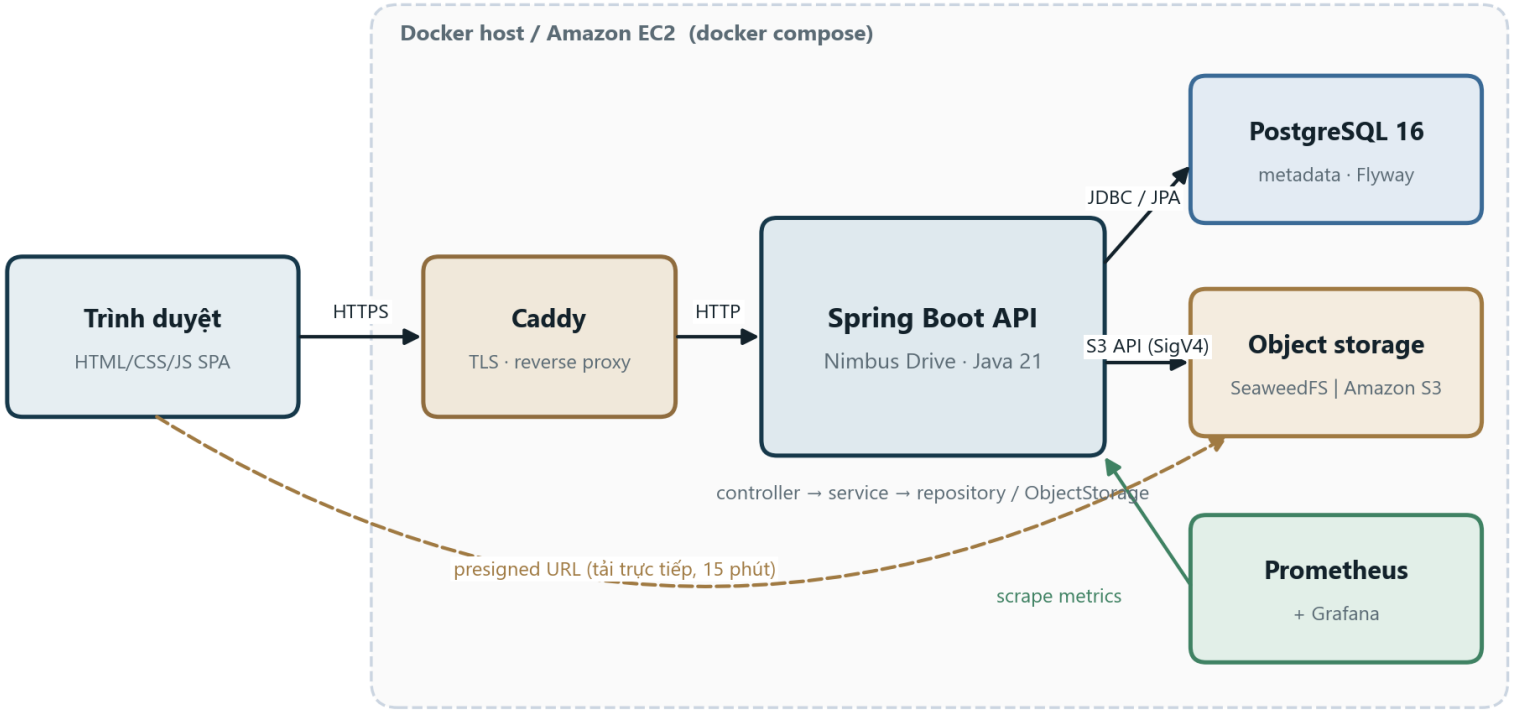
Đọc ngay sau khi ghi luôn thấy dữ liệu mới (từ 12/2020)

8

CORS + Lifecycle

Cho phép GET/HEAD từ trình duyệt; hủy multipart dở sau 7 ngày

Kiến trúc: tách metadata và nội dung file



PostgreSQL

Người dùng, thư mục ảo, metadata, link chia sẻ

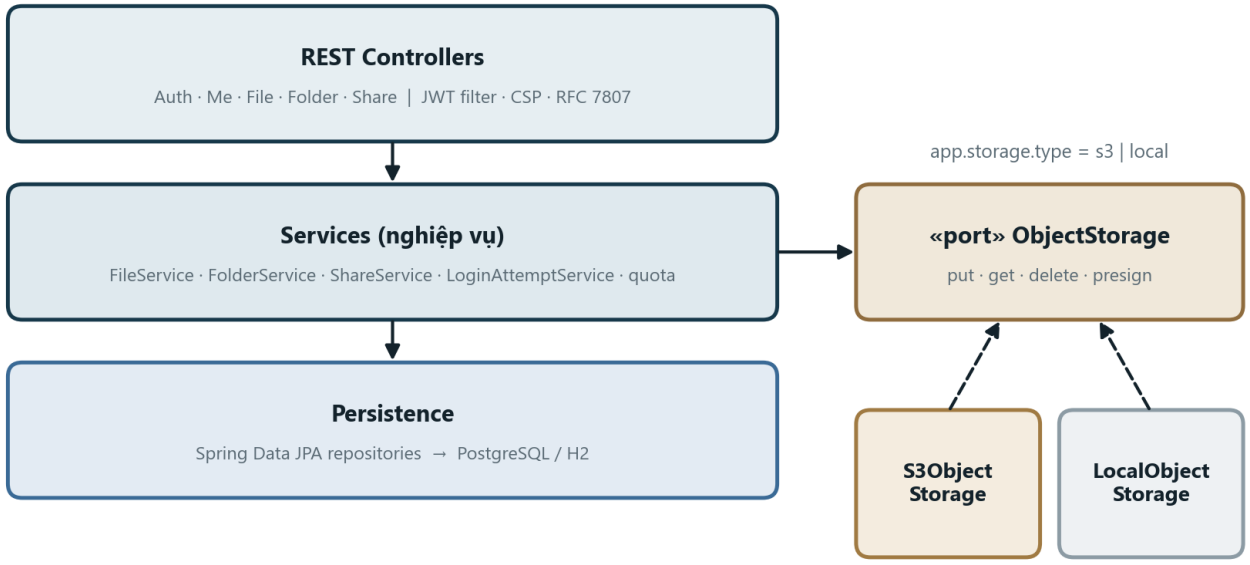
Object storage

Nội dung file, key phẳng owner/uuid

Presigned URL

Trình duyệt tải thẳng từ S3, giảm tải server

Ports & Adapters: một mã nguồn, nhiều nhà cung cấp S3



- Interface **ObjectStorage**: put · get · delete · presign
- **S3ObjectStorage** dùng AWS SDK v2 – chạy với AWS S3, SeaweedFS, Ceph, MinIO
- **LocalObjectStorage** cho dev/test không cần Docker
- Đổi backend chỉ bằng biến môi trường (12-factor)

Bài học thực tế

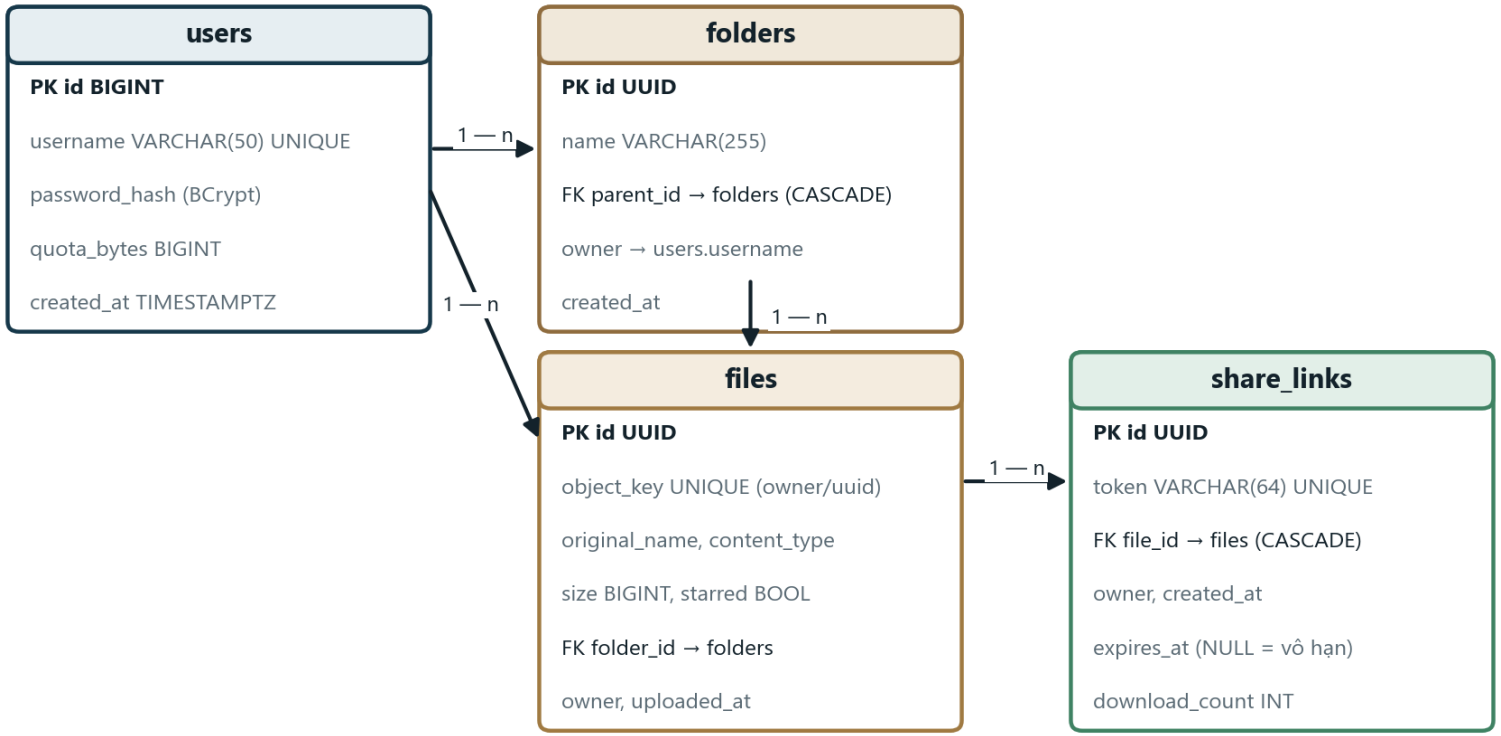
MinIO community ngừng phát hành bản dựng (410 Gone) → chuyển sang SeaweedFS mà không sửa dòng code nghiệp vụ nào.

Mã nguồn gọn: ~2.000 dòng Java, ~1.400 dòng giao diện

```
src/main/java/com/cloudfs/
  controller/  Auth Me File Folder Share
  service/     File Folder Share LoginAttempt
  storage/     ObjectStorage S3 Local
  security/    JwtService JwtAuthFilter
  config/      StorageConfig OpenApi
  web/         ApiExceptionHandler
src/main/resources/
  db/migration/V1__init.sql
  static/  index app share .html  css/ js/
src/test/  ApiFlowTest UnitTests *IT
deploy/  seaweedfs caddy monitoring terraform
Dockerfile  docker-compose(.aws).yml
.github/workflows/ci-cd.yml
```

Lớp	Công nghệ
Ngôn ngữ	Java 21 · Spring Boot 3.5
Bảo mật	Spring Security · JJWT · BCrypt
Dữ liệu	JPA/Hibernate 6 · PostgreSQL 16 · Flyway
Lưu trữ	AWS SDK v2 (S3Client, S3Presigner)
API docs	springdoc-openapi 2.10
Giám sát	Actuator · Micrometer · Prometheus · Grafana
Kiểm thử	JUnit 5 · MockMvc · Testcontainers · JaCoCo
Frontend	HTML/CSS/JS thuần, không bước build

Cơ sở dữ liệu: 4 bảng, quản lý bằng Flyway



- Thư mục dạng cây, parent_id **ON DELETE CASCADE**
- object_key duy nhất ánh xạ tới S3
- Token chia sẻ 192 bit, có hạn dùng
- Migration có phiên bản, chạy trên PostgreSQL & H2
- Index theo (owner, folder) cho truy vấn nhanh

SẢN PHẨM

Tính năng chính của Nimbus Drive

1

Tài khoản JWT

BCrypt, khóa tạm sau 5 lần sai

2

Upload kéo-thả

Nhiều file, thanh tiến độ, 100 MB/file

3

Thư mục lồng nhau

Breadcrumb, kéo file vào thư mục

4

Xem trước

Ảnh, PDF, video, audio, code

5

Chia sẻ link

Hết hạn 1h–30 ngày, đếm lượt, thu hồi

6

Presigned URL

Tải trực tiếp từ S3 15 phút

7

Quota & thống kê

1 GB/user, phân bố theo loại file

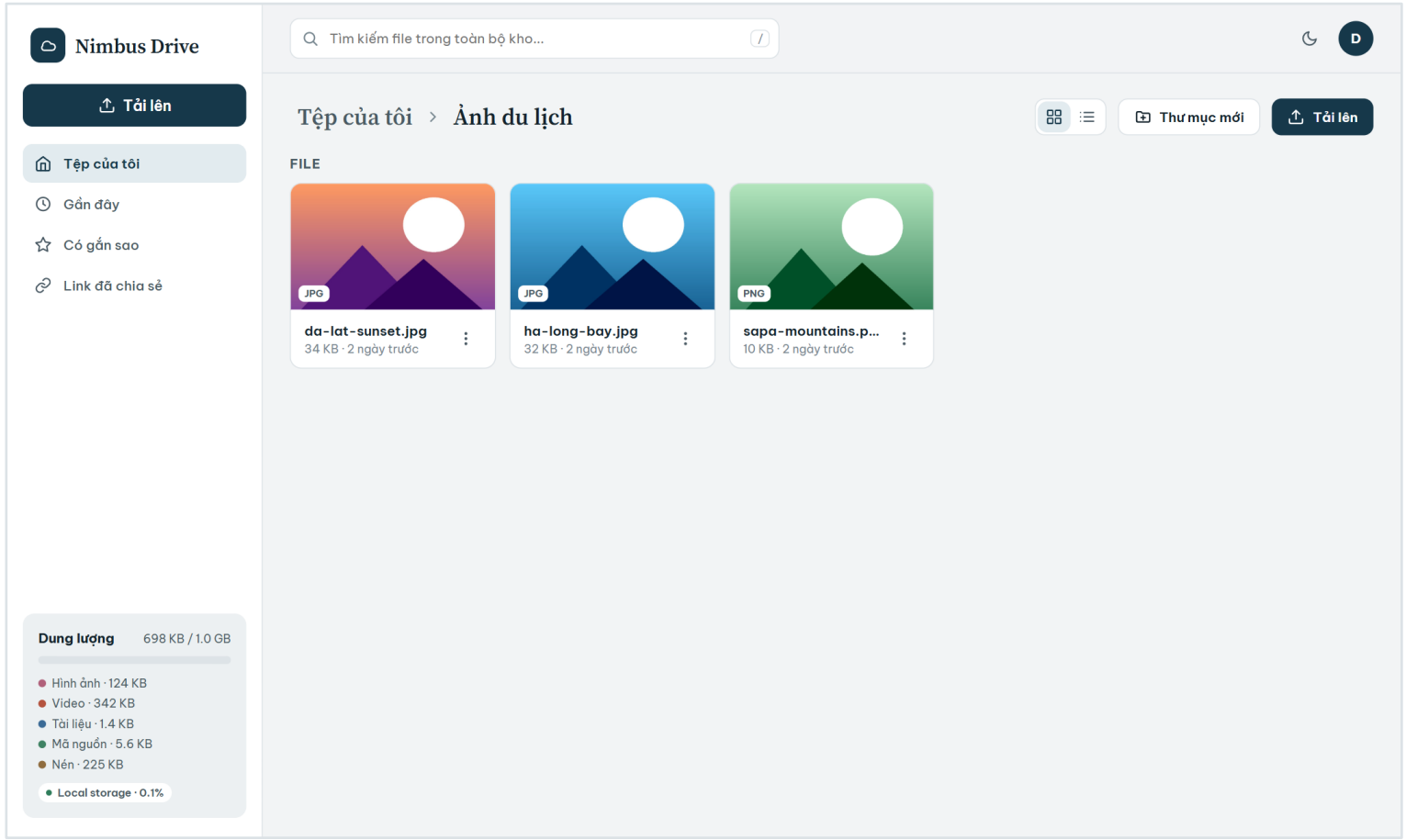
8

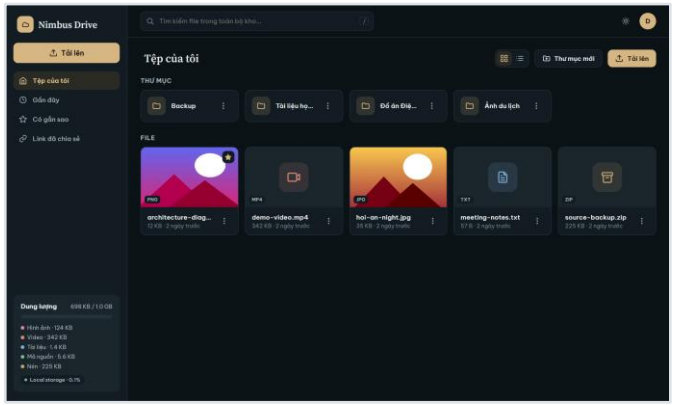
Trải nghiệm

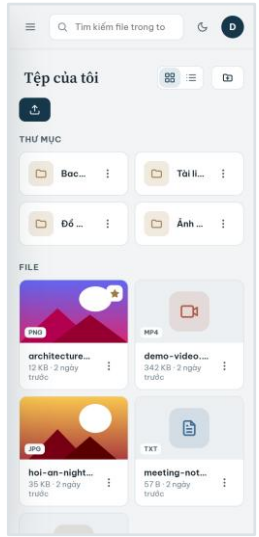
Tìm kiếm, gắn sao, dark mode, mobile

DEMO

Giao diện quản lý file



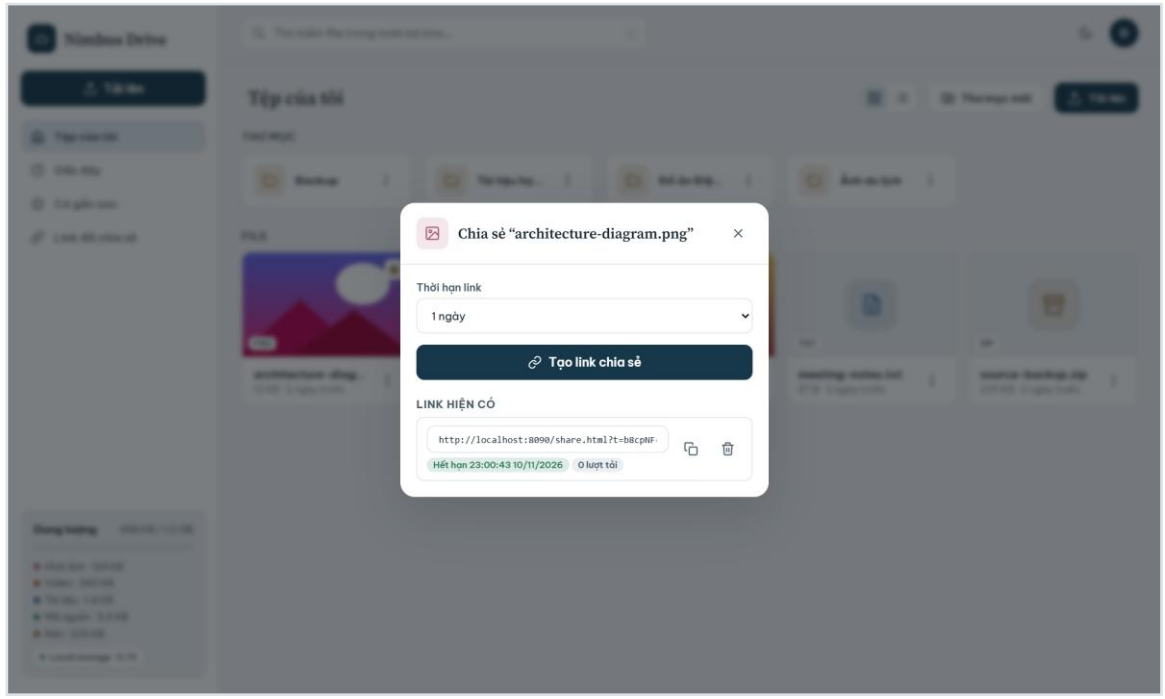




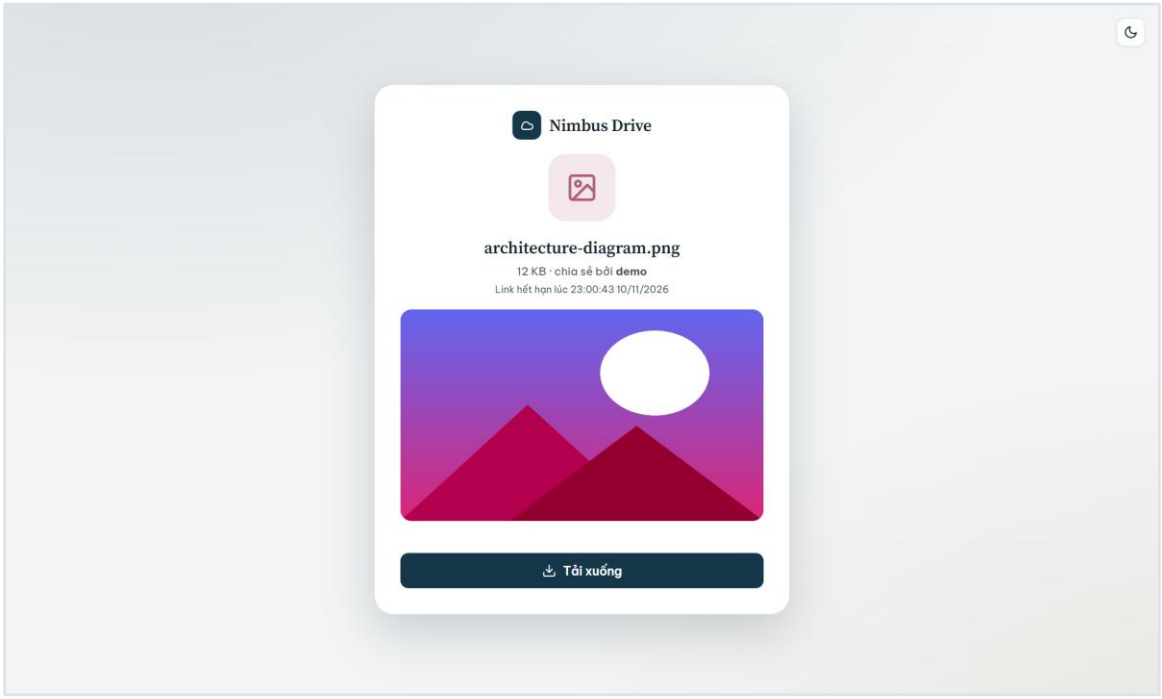
Lưới / danh sách · ảnh thu nhỏ · thống kê dung lượng · sáng/tối · responsive

DEMO

Xem trước và chia sẻ file



Tạo link có thời hạn, sao chép, thu hồi

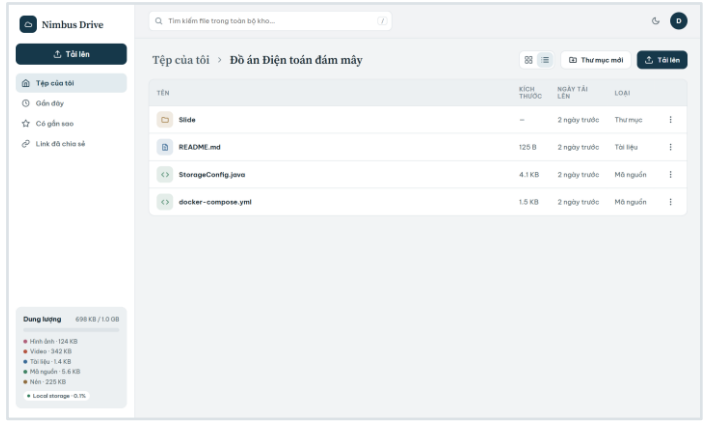


Khách tải file không cần đăng nhập

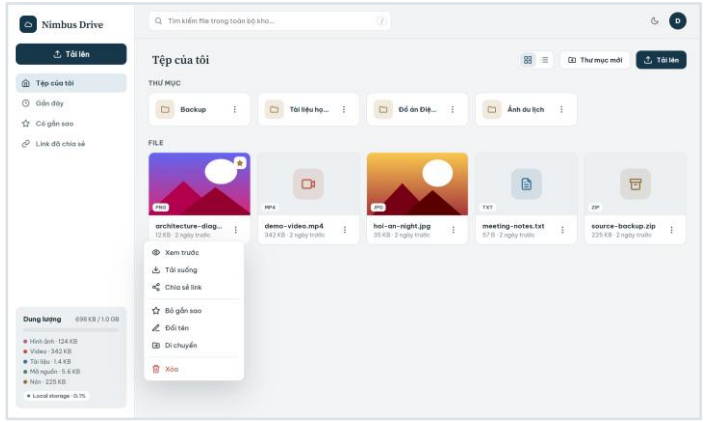
File HTML/SVG do người dùng tải lên luôn bị ép tải về (không hiển thị inline) để chống XSS

DEMO

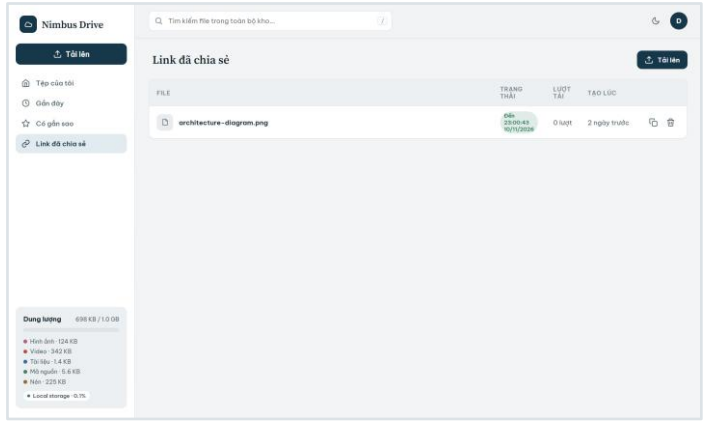
Danh sách, menu thao tác, link đã chia sẻ và Swagger



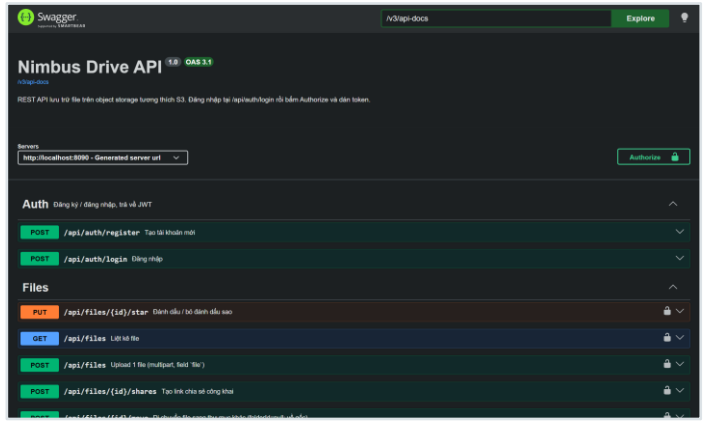
Chế độ danh sách + breadcrumb thư mục lồng nhau



Menu: xem, tải, đổi tên, sao, chia sẻ, Link S3

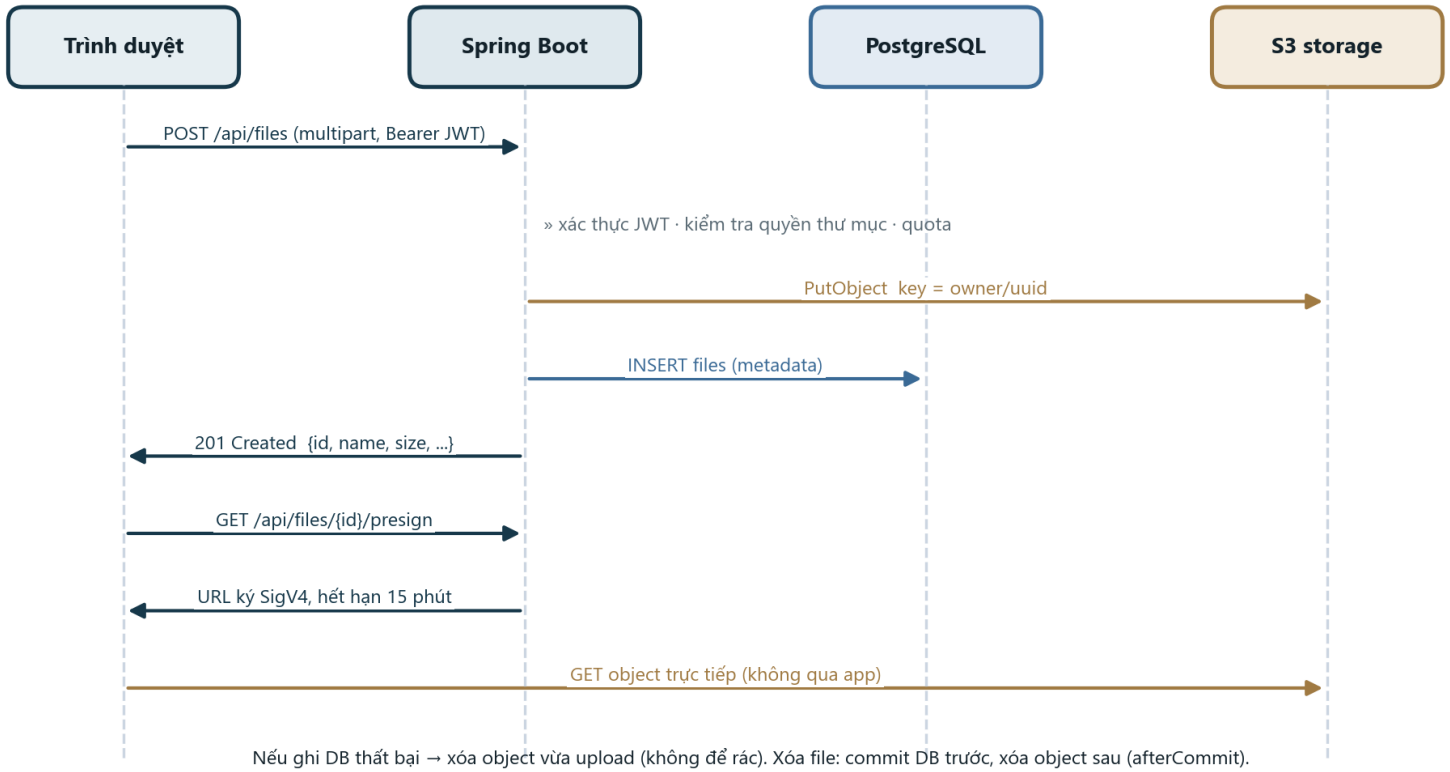


Quản lý link đã chia sẻ: lượt tải, hạn dùng, thu hồi



Swagger UI: tài liệu API tương tác

Luồng upload và tải qua presigned URL



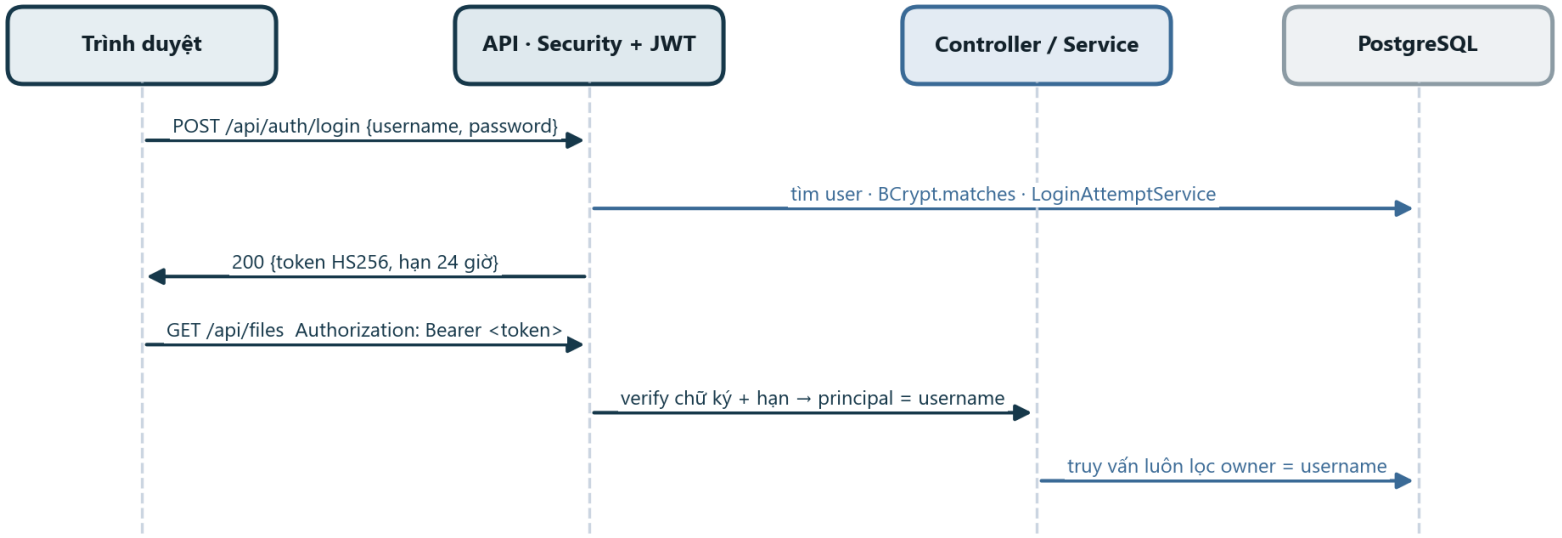
1. **Xác thực & kiểm tra quota**
2. **PutObject lên S3 trước**
3. **Ghi metadata; lỗi → xóa object**
4. **Xóa: commit DB rồi mới xóa object**

Hai cách phục vụ file: qua server và presigned URL

Tiêu chí	Tải qua máy chủ ứng dụng	Presigned URL tới S3
Đường đi dữ liệu	S3 → app → trình duyệt	S3 → trình duyệt (trực tiếp)
Tải cho app	Cao: app stream toàn bộ file	Gần như bằng 0
Kiểm soát quyền	Mỗi request qua JWT filter	Quyền nằm trong URL, hết hạn 15 phút
Yêu cầu hạ tầng	Chỉ mở cổng app	Trình duyệt thấy S3 + cấu hình CORS
Dùng trong đề tài	Tải xuống, xem trước, link chia sẻ	Menu “Link S3 tạm thời”, API /presign

Backend local (dev) không có S3 → /presign trả 501; có test kiểm tra hành vi này

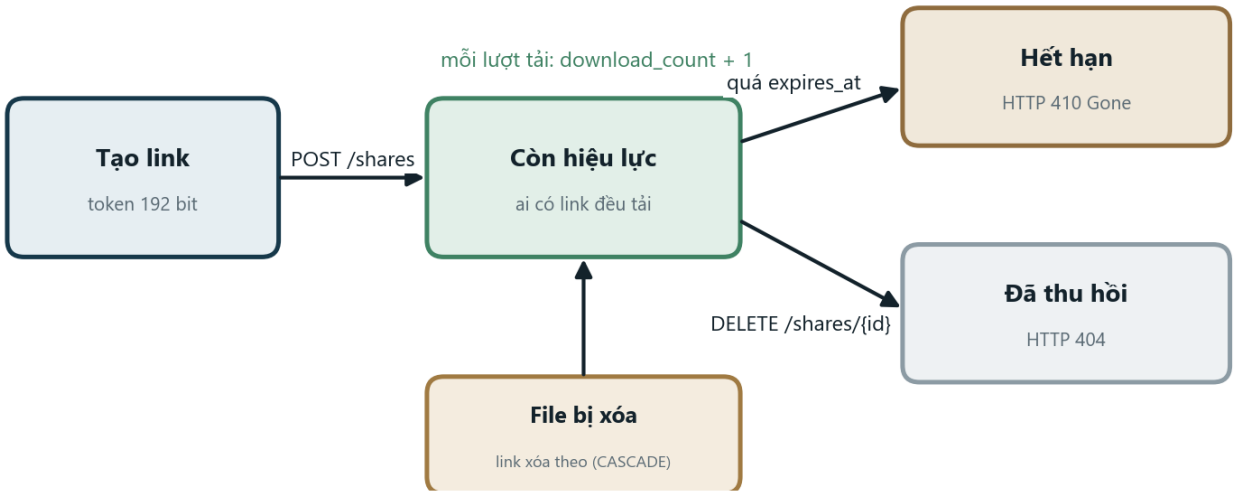
Xác thực JWT: stateless, mọi truy vấn lọc theo owner



Stateless: không có session phía server → scale ngang dễ; token sai/hết hạn → 401.

- Token **HMAC-SHA**, hạn **24 giờ**, khóa ≥ 32 ký tự từ biến môi trường
- Không lưu session → chạy nhiều instance dễ
- Sai **5 lần** → khóa **5 phút** (HTTP 429)
- Tài nguyên người khác → **404**, không lộ sự tồn tại
- BCrypt: có muối, cố ý chậm

Vòng đời của link chia sẻ công khai



Trang công khai chỉ trả tên, kích thước, loại file; không lộ object key hay tài khoản khác.

- Token **192 bit** (SecureRandom, Base64URL): không thể đoán
- Hạn: 1 giờ · 1 ngày · 7 ngày · 30 ngày · vô hạn
- Thu hồi bất cứ lúc nào; xóa file thì link xóa theo (CASCADE)
- Trang công khai chỉ trả tên, kích thước, loại file
- Hết hạn → **410**, thu hồi → **404**

BẢO MẬT

Bảo mật nhiều lớp

Xác thực

JWT HS256 · BCrypt · khóa 5 phút sau 5 lần sai (429)

Phân quyền

Mọi truy vấn lọc theo owner; tài nguyên người khác → 404

Chống XSS

CSP script-src 'self'; file HTML/SVG không hiển thị inline

Đầu vào

Làm sạch tên file, chặn path traversal, validate DTO

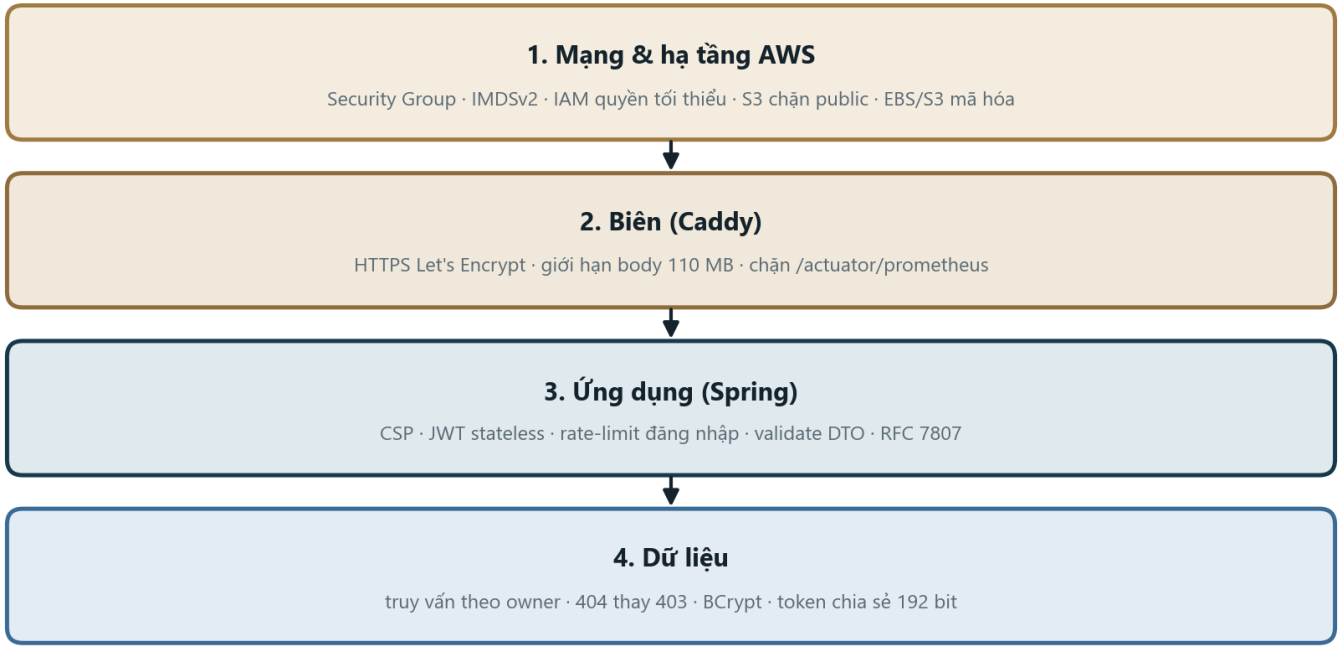
Hạ tầng AWS

IAM role quyền tối thiểu, IMDSv2, không lưu access key

Dữ liệu

S3 chặn public, mã hóa SSE, versioning; HTTPS qua Caddy

Phòng thủ nhiều lớp: bốn vòng bảo vệ



Kẻ tấn công phải vượt qua cả bốn lớp; một lớp hỏng không làm lộ toàn bộ hệ thống.

Chống XSS ở hai tầng

- Giao diện: CSP **script-src 'self'**, không inline script, escape dữ liệu khi render
- File người dùng: chỉ ảnh/PDF/text/video/audio được xem inline; **HTML, SVG ép tải về**
- Response file kèm CSP **sandbox** + default-src 'none'
- Tên file bị làm sạch: bỏ đường dẫn, ký tự điều khiển

Container hóa với Docker & Docker Compose

```
# Stage 1: build (JDK + Maven, cache .m2)
FROM maven:3.9-eclipse-temurin-21 AS build
RUN mvn package && java -Djarmode=tools \
    -jar app.jar extract --layers

# Stage 2: runtime (chỉ JRE)
FROM eclipse-temurin:21-jre
COPY dependencies/ application/ ./
USER appuser        # không chạy root
HEALTHCHECK curl /actuator/health/liveness
```

app

Spring Boot · chờ DB & S3 healthy

postgres

PostgreSQL 16 · volume

seaweedfs

S3 API :8333 tự host

caddy

HTTPS tự động (profile proxy)

prometheus + grafana

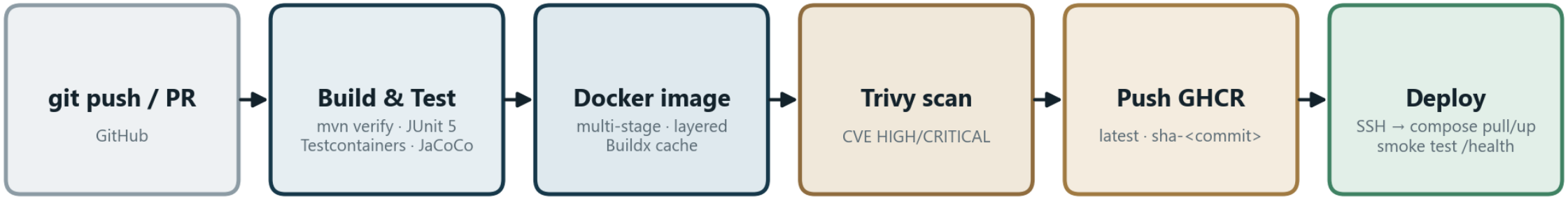
Giám sát (profile monitoring)

Cấu hình 12-factor: đổi môi trường chỉ bằng biến

Biến	Mặc định	Ý nghĩa
S3_ENDPOINT	localhost:8333	Trống = Amazon S3 thật; có giá trị = S3 tự host
S3_ACCESS_KEY / SECRET	nimbus…	Trống trên AWS → dùng IAM instance profile
S3_PATH_STYLE	true	true cho SeaweedFS/MinIO, false cho Amazon S3
S3_PUBLIC_ENDPOINT	= endpoint	Địa chỉ trình duyệt thấy, để ký presigned URL
STORAGE_TYPE	s3	s3 hoặc local (thư mục, chỉ dev)
JWT_SECRET	phải đổi	Khóa ký token ≥ 32 ký tự; Terraform sinh ngẫu nhiên
DEFAULT_QUOTA_BYTES	1 GB	Hạn mức mỗi người dùng
MAX_FILE_SIZE	100 MB	Kích thước tối đa mỗi file

Pipeline CI/CD tự động với GitHub Actions

GitHub Actions · .github/workflows/ci-cd.yml



Fail ở bất kỳ bước nào → dừng pipeline; production luôn chạy image đã qua test, gắn tag theo commit (rollback = deploy lại sha cũ).

3 job: test → image → deploy

2 tag: latest + sha-<commit>

90s smoke test sau deploy

CI/CD chi tiết: cấu hình và thực hành an toàn

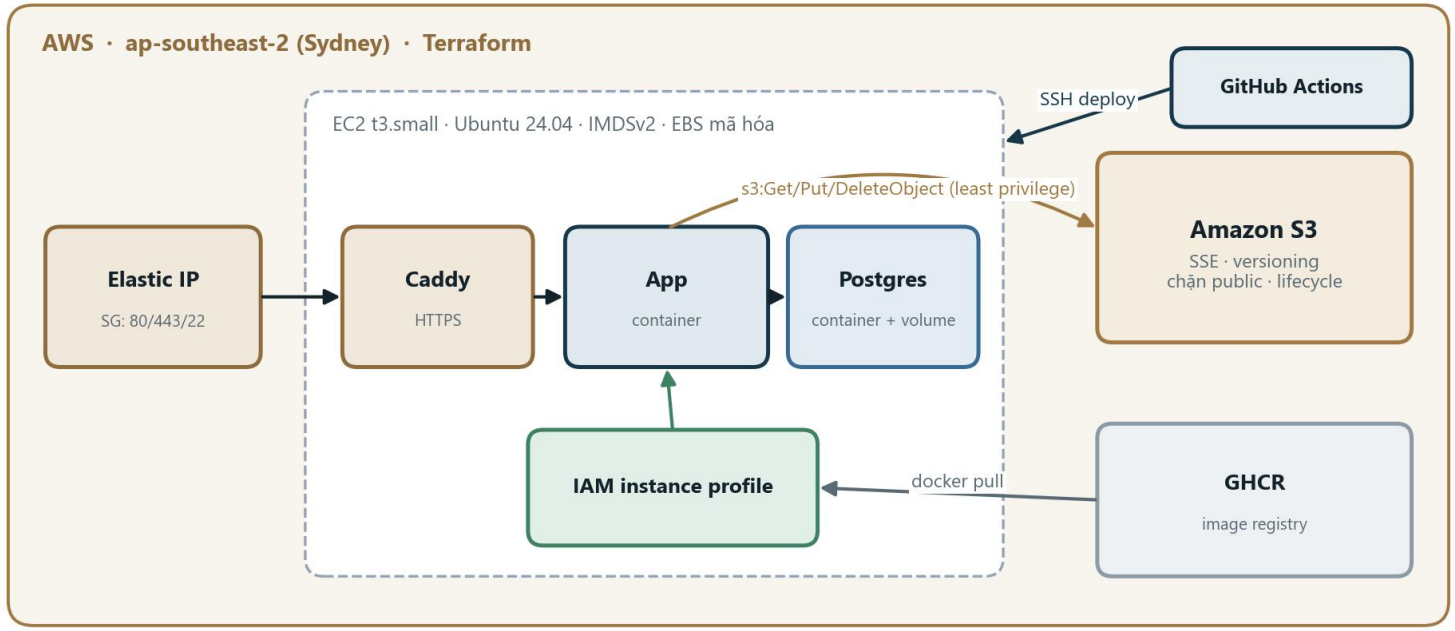
Cấu hình GitHub cho deploy

Tên	Mục đích
DEPLOY_ENABLED	true = bật job deploy
APP_URL	URL môi trường production
DEPLOY_HOST	Elastic IP của EC2
DEPLOY_USER	ubuntu
DEPLOY_SSH_KEY	Khóa riêng, không commit
GITHUB_TOKEN	pull/push GHCR

Thực hành an toàn trong pipeline

- **concurrency** hủy lần chạy cũ khi có push mới
- Action Trivy **ghim theo SHA**, không dùng tag
- Bước kiểm tra **skipped = 0**: integration test bị bỏ qua thì pipeline đỏ
- Quyền tối thiểu: job image ghi packages, job deploy chỉ đọc
- Environment **production** có thể bật duyệt thủ công
- Cache Buildx (type=gha) rút ngắn thời gian build
- Báo cáo test + JaCoCo lưu 14 ngày

Triển khai trên AWS bằng Terraform



- **S3**: chặn public, SSE-AES256, versioning, lifecycle, CORS
- **IAM role** chỉ Get/Put/Delete trên 1 bucket
- **EC2** Ubuntu 24.04, IMDSv2, EBS mã hóa, Elastic IP
- user_data cài Docker, sinh secret ngẫu nhiên
- Đã chạy thật ở Sydney: upload, S3, presigned URL, chia sẻ đều đạt

Quy trình triển khai từ đầu đến cuối: sáu bước

1

Đăng nhập AWS

aws login, tạo key pair EC2

2

Khai báo biến

terraform.tfvars: key_name, app_image, region

3

terraform apply

S3, IAM, Security Group, EC2, Elastic IP

4

Cấu hình GitHub

Secrets + DEPLOY_ENABLED=true

5

git push main

Test → image → Trivy → GHCR → deploy

6

Kiểm tra

/actuator/health = UP, đăng ký và upload thử

Rollback: chạy lại workflow ở commit cũ (tag sha-<commit>) · Dọn dẹp: terraform destroy

Giám sát: Prometheus + Grafana, 8 chỉ số chính

1

Uptime

up của job nimbus-drive

2

Request / giây

http_server_requests

3

Độ trễ p95

histogram phân vị 95

4

Lỗi 5xx (5 phút)

tỷ lệ lỗi phía server

5

File đã upload

nimbus.files.uploaded

6

Bytes upload

nimbus.files.upload.bytes

7

JVM heap

jvm_memory_used_bytes

8

Kết nối DB

hikaricp_connections_active

Scrape 15 giây qua /actuator/prometheus (Caddy chặn từ Internet) · Docker healthcheck: /actuator/health/liveness

KẾT QUẢ

Kết quả kiểm thử

22/22	~80%	22	5
ca test đạt trên CI	độ phủ dòng (JaCoCo)	REST endpoint	service container

Unit test	Tên file, phân loại, token, khóa đăng nhập, JWT
API test (MockMvc + H2)	Luồng đầy đủ: cô lập user, quota, thư mục đệ quy, chia sẻ, chống XSS
Integration (Testcontainers)	PostgreSQL 16 và SeaweedFS S3 thật – đạt trên GitHub Actions

KẾT QUẢ

22 ca kiểm thử: từ unit đến hệ thống thật

Lớp test	Ca	Điều được kiểm chứng
JwtServiceTest	3	Token đúng / sai khóa / hết hạn
UnitTests	4	Làm sạch tên file, phân loại, token chia sẻ, khóa đăng nhập
ApiFlowTest (MockMvc + H2)	11	401, validate, 409, khóa 429, vòng đời file, cô lập user, quota 413, thư mục đệ quy, chia sẻ, chống XSS, presign 501, health
PostgresMigrationIT	1	Flyway + Hibernate trên PostgreSQL 16 thật
S3ObjectStorageIT	3	Put/Get/Delete, presigned URL không cần credential, tạo bucket idempotent (SeaweedFS thật)

22/22 đạt trên GitHub Actions

18 chạy được mọi máy

4 cần Docker (Testcontainers)

KẾT QUẢ

Sự cố thật và bài học rút ra

Sự cố	Nguyên nhân gốc	Khắc phục / bài học
MinIO không tải được	Bản community ngừng phát hành (410 Gone)	Chuyển SeaweedFS chỉ bằng cấu hình
CI xanh giả	Testcontainers 1.19 bỏ qua 4 ca integration	Nâng 1.21.4 + bước kiểm tra skipped = 0
Không lấy được credential IAM	IMDSv2 hop limit = 1, container cần 2	http_put_response_hop_limit = 2
Không pull được image GHCR	Job deploy thiếu quyền packages: read	Thêm quyền + docker login bằng GITHUB_TOKEN
Trivy: 65 lỗ hổng HIGH/CRITICAL	Spring Boot 3.3 + thư viện cũ	Lên Boot 3.5.16, override phiên bản, apt upgrade → còn 2
Giao diện thiếu nhất quán	Bộ màu indigo – hồng, thiếu phân cấp	Design token: mực – sâm panh, serif, sáng/tối

KẾT LUẬN

Sáu quyết định thiết kế và đánh đổi

Vấn đề	Chọn	Đánh đổi
Nơi lưu file	Object storage chuẩn S3	Độ trễ cao hơn ổ đĩa cục bộ
Thư mục	Thư mục ảo trong PostgreSQL	Xóa đệ quy xử lý ở tầng service
Xác thực	JWT stateless	Khó thu hồi token trước hạn
Thứ tự ghi	S3 trước DB; xóa DB trước S3	Có thể sót object mồ côi nếu app sập
Frontend	JavaScript thuần, không build	Mã giao diện dài hơn framework
Hạ tầng AWS	1 EC2 + S3 qua Terraform	Không Multi-AZ; DB là điểm lỗi đơn

KẾT LUẬN

Chi phí vận hành trên AWS và cách tối ưu

Thành phần	Cách tính	Tối ưu
EC2 t3.small	Theo giờ chạy	Dừng máy khi không demo
EBS gp3 20 GB	GB-tháng, tính cả khi máy dừng	Xóa khi terraform destroy
IPv4 công khai / EIP	Tính phí theo giờ từ 02/2024	Giải phóng khi không dùng
Amazon S3	GB lưu + request + dữ liệu ra	Lifecycle; presigned URL giảm lưu lượng
GitHub Actions + GHCR	Trong hạn mức repo công khai	Cache Buildx

Không nêu đơn giá vì thay đổi theo vùng và thời điểm: tính bằng AWS Pricing Calculator. Biện pháp hiệu quả nhất: hạ tầng dưới dạng mã → dựng và hủy bằng một lệnh.

Hạn chế và hướng phát triển

Hạn chế

- Upload qua server, giới hạn 100 MB/file
- Rate-limit lưu RAM, chưa dùng chung nhiều instance
- PostgreSQL chung EC2, chưa backup tự động
- Còn 2 lỗ hổng cần Spring Boot 4; AWS chưa có HTTPS

Upload trực tiếp S3

Presigned PUT + multipart, file nhiều GB

Thùng rác & phiên bản

Xóa mềm, khôi phục qua S3 versioning

Managed services

RDS Multi-AZ, ECS Fargate, CloudFront

Bảo mật nâng cao

OAuth2 Google, 2FA, quét virus, Redis

Kết luận và bài học kinh nghiệm

1	**Chuẩn mở là bảo hiểm**	Bám chuẩn S3 + port/adapter → đổi MinIO sang SeaweedFS, local sang AWS không sửa nghiệp vụ
2	**Thứ tự thao tác > giao dịch phân tán**	Với hai kho dữ liệu, chọn đúng thứ tự ghi/xóa đơn giản và hiệu quả
3	**Kiểm thử trên môi trường giống thật**	Lỗi hop limit và Testcontainers bị bỏ qua chỉ lộ khi chạy trên AWS / CI
4	**Bảo mật là quá trình liên tục**	Trivy: 65 → 2 lỗ hổng sau nâng cấp; cần quy trình cập nhật định kỳ
5	**Hạ tầng dưới dạng mã**	Dựng, kiểm thử, hủy cả môi trường AWS bằng một lệnh

Cảm ơn thầy cô và các bạn đã lắng nghe

Hỏi & đáp · Demo trực tiếp

`docker compose up -d --build`	`http://localhost:8080`	`/swagger-ui.html`	`./mvnw verify`
Chạy toàn bộ hệ thống	Giao diện web	Tài liệu API	Chạy kiểm thử